An Ascon-AEAD128 Hardware Accelerator

Model-driven design on a fully open RISC-V SoC, validated on silicon

Ramin Mohammadi Masoudi

Politecnico di Milano

NEORV32 RISC-V SoC · Tang Nano 20K · fully-open toolchain

Outline

In one sentence

A hardware accelerator for the NIST lightweight-cryptography standard, *generated* from a verified model, integrated into an open RISC-V SoC, and measured on real hardware.

Motivation

Why this project

- **Ascon** is the NIST standard for lightweight authenticated encryption (SP 800-232) — built for sensors, embedded controllers, IoT nodes.
- Its permutation works on five **64-bit** words. On a **32-bit** microcontroller every 64-bit rotation costs several instructions.
- So Ascon in software is slowest exactly on the devices it targets. That is the gap hardware should close.

Four objectives

1. Hardware Ascon that matches the standard, provably
2. A clean C interface on a RISC-V SoC
3. An open, reproducible toolchain
4. Validate on real hardware and *measure*

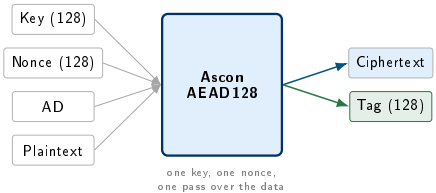
The algorithm

What authenticated encryption has to provide

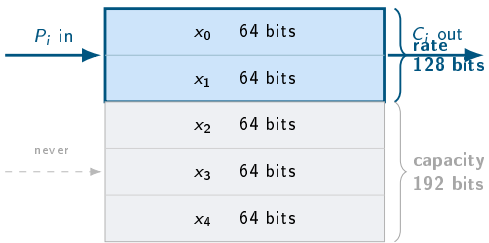
An AEAD scheme takes a **key**, a **nonce**, some **associated data** and a **message**, and must deliver three things at once:

1. **Confidentiality** — the message is unreadable without the key.
2. **Integrity & authenticity** — *any* modification is detected. This is the **tag**: 128 bits that verify or don't.
3. **Associated data** — authenticated but *not* encrypted. A packet header you must be able to read, but nobody may alter.

Encryption alone is not enough: without a tag, an attacker can flip bits in the ciphertext and you would never know.



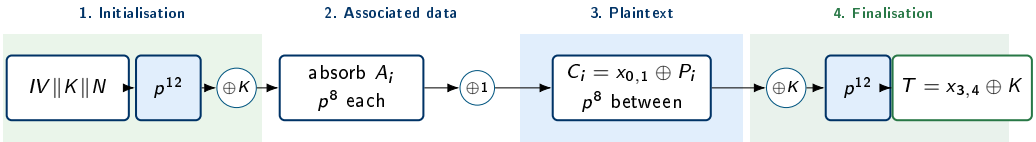
Ascon is a sponge: one 320-bit state



- **320 bits**: five 64-bit words.
- Data **only touches the rate** — the top 128 bits. XOR a block in; the rate becomes the ciphertext.
- The **capacity** is never read or written by data. It carries the security.
- Between blocks the permutation stirs all 320 bits.

Why this matters for hardware
Encryption, hashing and XOF share the *same* state and *same* permutation — only the IV and padding differ. **One permutation is the entire engine.**

Ascon-AEAD128: four phases



Parameters, from the standard

`IV = 0x00001000808c0001`

rate **16 bytes** $a = 12$ $b = 8$ tag **128 bits**

Associated data is authenticated but *not* encrypted — a header you must not tamper with, but need to read in clear.

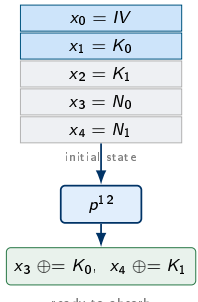
The **tag** is what makes this *authenticated* encryption: flip one bit of the ciphertext and it stops verifying. I will show that on the board.

Phases 1 & 2: initialisation, associated data

1. Initialisation

- Load directly: $x_0 \leftarrow IV$, $x_1 \| x_2 \leftarrow K$, $x_3 \| x_4 \leftarrow N$.
- Run p^{12}, then XOR the key in *again*: $x_3 \oplus= K_0$, $x_4 \oplus= K_1$.

Why twice? The key enters before *and* after a full 12-round permutation — so the state depends on it in a way an attacker cannot unwind.



2. Associated data

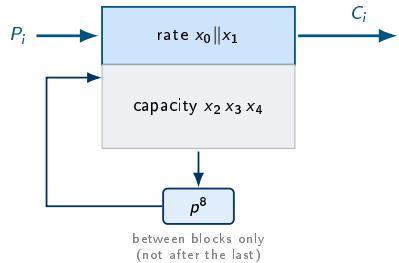
- Per 16-byte block: $x_0 \| x_1 \oplus= A_i$, then p^8.
- Padding: append 0x01, then zeros — and if the AD is already a multiple of the rate, absorb an **extra all-padding block**.
- **Domain separation**: $x_4 \oplus= 1 \ll 63$ — the last bit of the 320-bit state.

Phases 3 & 4: plaintext, finalisation

3. Plaintext

- XOR the block into the rate — and the rate *becomes* the ciphertext: $C_i = (x_0 \| x_1) \oplus P_i$.
- Then p^8 *between* blocks — but **not** after the last; finalisation follows immediately.

The elegance: absorbing the plaintext and producing the ciphertext are *the same XOR*. Decryption differs only in which value you keep.



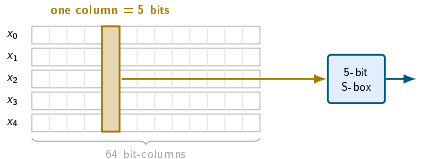
4. Finalisation

- $x_2 \oplus= K_0$, $x_3 \oplus= K_1$, then p^{12}.
- Tag: $T = (x_3 \oplus K_0) \| (x_4 \oplus K_1)$.

The tag depends on the key, the nonce, *every* AD byte and *every* message byte. Change one and it fails.

Inside the permutation: three steps, twelve times

1. **Add round constant** p_C
   - $x_2 \oplus= c_r$ — a different constant each round.
   - Breaks the symmetry between rounds.
2. **Substitution** p_S
   - A **5-bit S-box** — the *only* non-linear step in the cipher.
   - Applied to a **bit-column**: one bit from each of $x_0 \ldots x_4$.
   - **64 columns**, completely **independent** of one another.
3. **Linear diffusion** p_L
   - Each word XORed with two rotations of itself — spreading each column's result across the word, so next round the columns all differ.



The p_L rotation constants, from the standard:

$$
\begin{aligned}
x_0 &\oplus= (x_0 \ggg 19) \oplus (x_0 \ggg 28) \\
x_1 &\oplus= (x_1 \ggg 61) \oplus (x_1 \ggg 39) \\
x_2 &\oplus= (x_2 \ggg 1) \oplus (x_2 \ggg 6) \\
x_3 &\oplus= (x_3 \ggg 10) \oplus (x_3 \ggg 17) \\
x_4 &\oplus= (x_4 \ggg 7) \oplus (x_4 \ggg 41)
\end{aligned}
$$

The design lever: one choice spans the whole space

The 64 S-boxes are independent. Build all 64 and do a round per cycle — or build *one* and take 64 cycles per round. That single choice trades area against time, and spans the entire architecture space.

Architecture	S-boxes / cycle	Cycles / p^{12}	Natural target
bit-serial	fraction of one	1 561	smallest possible ASIC
column-serial, 1 col	1	793	minimum-area ASIC
column-serial, 8 cols	8	121	small ASIC
1 round / cycle	**64**	**13**	**this board**
2 rounds / cycle	128	7	FPGA, more area
4 rounds / cycle	256	4	FPGA, throughput
8 rounds / cycle	512	3	large FPGA
fully pipelined	768 (12 stages)	1 *	maximum throughput

* one result per cycle once the 12-stage pipeline is full. All cycle counts **measured** in simulation, not estimated.

All seven are generated from the model — and verified against it

Each is emitted by the generator and replayed against the model: **126 random trials each, zero mismatches**. A **120**× span in latency from one specification. I realised the highlighted one on silicon.

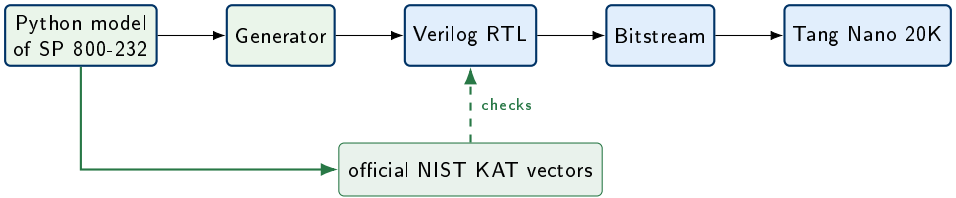
Design decisions

The decisions that shaped the project

Question	Decision	Why
Where does "correct" come from?	A typed Python model of SP 800-232, passing the official KATs	The model *generates* the RTL *and* checks it — one definition of correct
How is it coupled to the CPU?	Bus peripheral on XBUS, 0x9000_0000	Stock CPU, no pipeline changes; portable to any Wishbone master
How fast a permutation?	1 round / cycle	Fits the GW2AR-18 — and compute turns out to be 2 % of the time
How does data get in?	MMIO, 32-bit word at a time	The simplest plane that could be *fully verified* for first silicon
How much data per call?	Bounded: 32 B AD + 32 B message	Bounded buffers are exhaustively checkable in simulation
Which toolchain?	GHDL, Yosys, nextpnr, Apicula — pinned by Nix	Reproducible, no vendor lock-in
Which ISA?	`rv32i` (M disabled)	Needed to fit; Ascon has no multiplies, so it costs nothing

Each row is a trade-off taken deliberately. The one I would revisit is the data plane — and I show the measurement that says so.

Model-first: the central decision



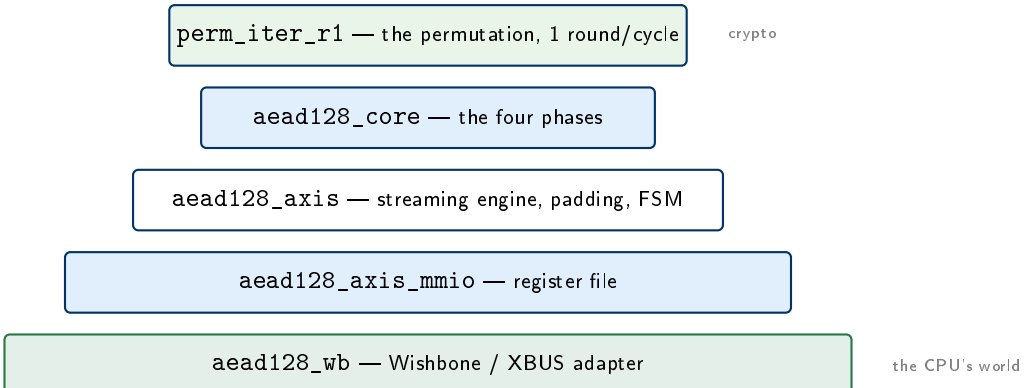
One source of truth

The model is both the **source** of the hardware and the **oracle** that checks it. The RTL and the test vectors descend from the same validated definition.

The consequence, borne out

Every bug that reached the board was an **integration** bug — a bus handshake, a protocol order. Not one was a cryptographic bug.

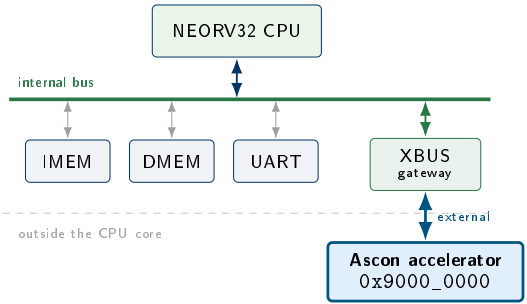
The accelerator: five layers, each checked alone



One job per layer, each replayed against the model in isolation — so a failure names its own layer. The bottom layer is where all the trouble was.

Integration: a peripheral, not a custom instruction

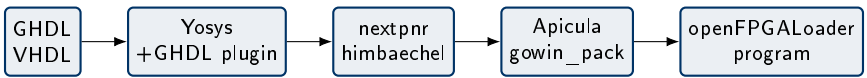
- CPU: NEORV32, `rv32i`, 27 MHz — completely **stock**.
- The accelerator hangs off the **external** bus (XBUS) at 0x9000_0000: no CPU pipeline changes, no custom instructions.
- The driver hides every register behind `encrypt()` / `decrypt()`.



Why not a custom instruction?

Tighter coupling would cut data movement — a real advantage, and where the future work points. But it welds the design to one pipeline. On a standard bus the interface is public, portable and testable.

A fully-open, reproducible toolchain



No vendor EDA at any step

Every stage from VHDL/Verilog to a programmed FPGA is open source, and the exact versions are pinned by a Nix flake. `nix flake check` runs the whole test suite from a clean checkout.

Verification

Four layers of verification

Layer	What is checked	Against
1	The Python model	the official NIST Known-Answer Tests
2	Every generated RTL module	the model (25 vectors per module)
3	The C driver	an emulated peripheral
4	End-to-end co-simulation	a NEORV32-faithful XBUS master

119 automated tests

Run from a clean checkout; reproduced by `nix flake check`. Layer 4 replays the driver's real register order through the bus adapter with NEORV32's exact pulsed-strobe timing — it exists *because* of the bug on the next slide but one.

From “simulation passes” to “correct on silicon”

Eight real bugs — each diagnosed from the board’s own evidence, not guesswork:

1. Read-only build environment
2. Toolchain name mismatch
3. Float-ABI (double vs. soft) mismatch
4. **Stack past end of RAM** → silent trap
5. Flaky USB debugger / shifting ports
6. **XBUS bus handshake**
7. FPGA fit (drop the M extension)
8. **START-first protocol order**

Evidence, not guessing

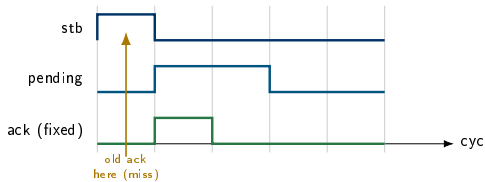
A processor panic with a precise fault address (0x9000 0000, then 0x9000 0044) named exactly which register — and which mechanism — had failed.

The key hardware bug: the bus handshake

NEORV32's XBUS:

- pulses the strobe for *one cycle*
- samples the acknowledge only in the *next* cycle

My first adapter: acknowledged *during* the strobe — the one cycle the CPU was not looking. ⇒ no ack seen → bus timeout → access fault at 0x9000_0000. **Fix:** a handshake FSM that latches the strobe, holds the access until the peripheral is ready, and acks in the cycle the CPU samples.



Against a naive testbench this passes. Against the real CPU it fails every time — so Layer 4 now models the CPU exactly.

Results

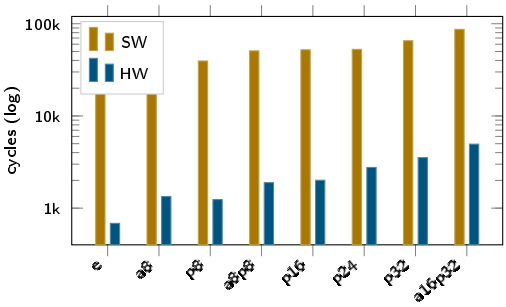
Correct on hardware, and 18–57× faster

All 8 cases correct on the board (`enc_ok`, `dec_ok`, `tag_valid`). Cycles at 27 MHz:

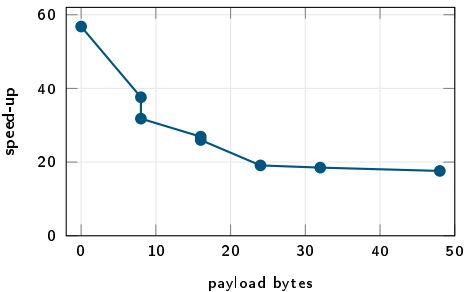
Case	AD	PT	SW	HW	Speed-up
`empty`	0	0	38 866	684	**56.8×**
`ad8`	8	0	50 205	1 335	37.6×
`pt8`	0	8	39 458	1 239	31.8×
`ad8_pt8`	8	8	50 797	1 890	26.9×
`pt16`	0	16	52 156	2 007	26.0×
`pt24`	0	24	52 748	2 763	19.1×
`pt32`	0	32	65 447	3 531	18.5×
`ad16_pt32`	16	32	86 984	4 948	17.6×

Cases probe the block structure: empty, AD only, PT only, both, and partial vs. full 16-byte rate blocks.

Results, visualised



cycles: software vs. hardware



speed-up falls as payload grows

Larger payloads ⇒ more word-at-a-time movement over MMIO ⇒ lower speed-up. The crypto itself is a handful of cycles.

Where the time actually goes

Simulating the AEAD core *alone* — same vectors, data already present — separates *crypto* from *waiting for data*:

Case	HW busy	Crypto	Waiting on I/O	Crypto %
`empty`	684	36	648	5.3 %
`pt16`	2 007	48	1 959	2.4 %
`pt32`	3 531	60	3 471	1.7 %
`ad16_pt32`	4 948	83	4 865	**1.7 %**
all 8 cases	**18 397**	**405**	**17 992**	**2.2 %**

The bottleneck is the interface, and it is not close

- Make the permutation **8× faster** (the 8-rounds/cycle core I already have): 17.6× → 17.8×. +0.2×.
- Remove the data-movement cost entirely (an upper bound, not a design): 17.6× → ∼**1050**×.

So I rebuilt the data plane — and measured it on silicon

The engine was *already* AXI4-Stream native. One line in the MMIO wrapper was the entire bottleneck:

```
wire s_tvalid = din_ctl_wr;
```

A streaming datapath, clocked by a **CPU store**.

The replacement

NEORV32's **SLINK** is AXI4-Stream at 32 bits — an exact match — and its **DMA** takes a constant destination address:

DMEM → DMA → SLINK → **Ascon** → SLINK → DMA → DMEM

The CPU writes key, nonce, lengths and START — then touches **no payload byte at all**.

	busy cycles		vs software	
Case	MMIO	**SLINK**	MMIO	**SLINK**
empty	684	**41**	56.8×	**948×**
ad8	1 335	**296**	37.6×	**170×**
pt8	1 239	**286**	31.8×	**138×**
pt16	2 007	**336**	26.0×	**155×**
pt32	3 531	**432**	18.5×	**152×**
ad16_pt32	4 948	**535**	17.6×	**163×**
total	**18 397**	**2 639**	**7.0× fewer**	

8/8 correct on the board. Same vectors, same counter.

Crypto share: **2.2%** → **15.3%**.

The new bottleneck — measured the same way

Simulation predicted 118 cycles for `ad16_pt32`; silicon gave 535. That gap is not noise. It is the DMA, and it fits a straight line.

Case	words	sim	silicon	gap
empty	0	41	41	**0**
ad8	2	55	296	241
ad8_pt8	4	60	333	273
pt24	6	69	380	311
pt32	8	87	432	345
ad16_pt32	12	118	535	417

$$\text{overhead} = 204 + 17.7 \times \text{words}$$

fits every case to within **2 cycles**

- **204 cycles** — programming and starting one DMA descriptor.
- **17.7 cycles/word** — NEORV32's DMA is a read-then-write engine, not a burst master.
- `empty` sends no words, so it has **zero** overhead — and lands exactly on the simulated 41.

So: the stream datapath is precisely as designed. The entire shortfall is the data mover.

Which is, once again, a measurement rather than a guess — and it names the next piece of work: a burst-capable master, not a faster permutation.

Faster than *what*? — being precise

The comparison

Software = `rdcycle` around a reference C Ascon-AEAD128 on the same `rv32i` core.
Hardware = the accelerator's busy-cycle counter, same 27 MHz.

Caveats, stated plainly:

- The software is a *reference*, not hand-optimised — an optimised version would narrow the gap (hardware still wins).
- The hardware counter excludes register setup and readback; the full end-to-end wall-clock number is lower, especially for `empty`.
- The $\sim$18$\times$ on real payloads is much closer to end-to-end than the $\sim$57$\times$ on the empty case.

Fair phrasing: **the hardware engine vs. reference software on the same SoC** — same platform, same clock, same counter.

Conclusions

Limitations & future work

Limitations — stated, not hidden

- **Bounded payload**: 32 B AD + 32 B message. Chosen so the length handling could be verified exhaustively.
- **No side-channel hardening.** No masking, no fault countermeasures, no characterisation. The tag compare is constant-time by construction; that is one channel, not a claim.
- **Key registers are readable.** Mirrored from the reference map and never hardened — a defect for a product.
- **Hash/XOF exist in the model only**, not on the board.
- **86.5% LUT utilisation** forced `RISCV_ISA_M` off.

Done since the measurement

- **The data plane is rebuilt and on silicon**: SLINK + DMA, **17.6**× → **163**×, 8/8 correct.

Next — and now measured, not guessed

- **A burst-capable master.** The DMA costs $204 + 17.7 \times$ words; that, not the permutation, is the remaining 85%.
- **Then** the faster permutation earns its area — at 15% crypto it finally moves the number.
- Arbitrary-length streaming; hash/XOF on hardware; a hardened key path.

Conclusion

What was achieved

- A **model-driven** Ascon-AEAD128 accelerator: one typed Python model, anchored to the official NIST vectors, that *generates* the RTL *and* verifies it.
- **Seven permutation architectures** from that one model, all verified — a **120**× latency span.
- **Silicon**, not simulation: a stock NEORV32 SoC on a Tang Nano 20K, 8/8 correct, entirely open toolchain.
- **119 automated tests**, reproducible with `nix flake check`.

What I would want you to take away

I measured before I optimised. The measurement said the crypto was **2.2%** of the time and the interface was **97.8%** — so a faster permutation was worth +0.2× and was not the answer.

So I rebuilt the data plane instead. On the board: **17.6**× → **163**×, and the crypto share rose from 2.2% to 15.3%.

And the new bottleneck is measured too: $204 + 17.7 \times$ words of DMA. Not a guess — a straight line through eight silicon data points.

Measure. Fix the thing the measurement names.
Measure again.